

ajoy

Options Trading Bot Developer Guide

A plain-English walkthrough of the codebase,
strategy logic, and configuration — for anyone
who knows the basics of Python.

v1 · May 2025

1 · What Is ajoy?

ajoy is a fully automated options-trading bot built in Python. It watches a list of stocks in real time, looks for specific technical setups, and places option orders when conditions are met — all without any manual intervention.

Think of it as a disciplined trader that never gets emotional, never skips a stop-loss, and follows the same rules on every single trade.

Tech Stack

FastAPI — the web framework that provides the dashboard and REST API. FastAPI is modern, fast, and uses Python type hints everywhere.

SQLAlchemy + SQLite — stores every trade, order, and position in a local database file (ajoy.db). No external database server needed.

APScheduler — runs two background jobs on a timer: one scans for new entries every 60 seconds, another manages open trades every 30 seconds.

Tradier API — the brokerage integration. Provides live market data (quotes, option chains, historical bars) and executes real orders. A sandbox environment is available for testing without real money.

Two Environments

Mode	API URL	Purpose
Sandbox	sandbox.tradier.com	Paper trading — no real money. Safe for testing.
Live	api.tradier.com	Real brokerage account. Orders execute for real.

2 · How It Works — The Trading Loop

Once started, ajoy runs two infinite loops in the background:

scan_for_entries() · runs every 60 seconds

For each ticker in your watchlist, this job:

1. Fetches the latest 1-min and 15-min OHLCV bars from Tradier.
2. Runs the bar data through all 7 entry gates (see Section 3).
3. If every gate passes, selects an option contract and places a BUY order via the Tradier API.
4. Saves the new trade to the database.

```
# app/services/strategy.py
async def scan_for_entries(db):
    for symbol in settings.watchlist:
        bars_1m = await get_bars(symbol, "1min", days=1)
        bars_15m = await get_bars(symbol, "15min", days=5)
        signal = check_entry_signal(bars_1m, bars_15m)
        if signal:
            contract = await select_option(symbol, signal.direction)
            if contract:
                await place_order(db, symbol, contract, signal)
```

manage_open_trades() · runs every 30 seconds

For each currently open trade, this job:

1. Fetches the latest mid-price of the option contract.
2. Applies all exit rules (hard stop, take-profit, trailing stop, VWAP break, trend reversal, end-of-day).
3. If an exit condition fires, places a SELL order and closes the trade.

```
# app/services/strategy.py
async def manage_open_trades(db):
    open_trades = await get_open_trades(db)
    for trade in open_trades:
        price = await get_option_price(trade.option_symbol)
        exit_reason = check_exit_signal(trade, price, bars_1m, bars_15m)
        if exit_reason:
            await close_trade(db, trade, price, exit_reason)
```

3 · The 7 Entry Gates

Every potential entry must pass all seven gates in order. If any gate returns False the trade is skipped and we wait for the next scan. The gates are called inside `check_entry_signal()`.

Layer	Name	What it checks
L1	EMA Trend	Price must be on the correct side of the 9-period EMA on 15-min bars. NEW: the last 2 bars must ALL
L2	VWAP Bounce	Price must have pulled back to within 0.2% of VWAP on the 1-min chart, then shown a bounce (2 con
L3	Volume Spike	Current 1-min bar volume must be at least 2× the 20-bar average — confirms real buying/selling press
L4	VWAP Slope	VWAP must not be moving sideways — requires the slope over the last 20 bars to exceed 0.05% thre
L5	Market Regime	SPY must agree with the trade direction. If the market is falling and we want to buy a CALL, the gate b
L6	IV Filter	The option's implied volatility must be below 150%. Extremely high IV means the option is overpriced
L7	Pre-entry Guards	Checks: not in earnings blackout, not in cooldown after a recent loss, not exceeded max losses for this

Layer 1 — EMA Trend + Consecutive Bars

```
def is_ema_trend_confirmed(bars_15m, direction, period=9, n=2):
    """
    Return True only if the last `n` 15-min bars ALL closed on the
    correct side of the EMA. Blocks entries on a freshly-flipped EMA.
    """
    if direction not in ("bullish", "bearish"):
        return False
    if n <= 0:
        return True # check disabled

    closes = [b.close for b in bars_15m]
    ema_vals = calculate_ema(closes, period)
    valid = [(c, e) for c, e in zip(closes, ema_vals) if e == e]

    if len(valid) < n:
        return False

    for close, ema in valid[-n:]:
        eps = ema * 0.00005 # tiny epsilon to avoid floating-point ties
        if direction == "bullish" and not (close > ema + eps):
            return False
        if direction == "bearish" and not (close < ema - eps):
            return False
    return True
```

Layer 2 — VWAP Bounce

```
def check_bounce_confirmation(bars_1m, direction, n=2):
    """
```

```

Require the last `n` completed 1-min bars to close on the
correct side of VWAP – confirms the bounce is real, not a wick.
"""
vwap = calculate_vwap(bars_1m)
completed = bars_1m[:-1] # exclude the still-forming bar
for bar in completed[-n:]:
    if direction == "bullish" and bar.close <= vwap:
        return False
    if direction == "bearish" and bar.close >= vwap:
        return False
return True

```

Layer 3 — Volume Spike

```

def has_volume_spike(bars_1m):
    recent_vol = bars_1m[-1].volume
    avg_vol = mean([b.volume for b in bars_1m[-21:-1]])
    return recent_vol >= avg_vol * settings.volume_spike_multiplier

```

Layer 5 — Market Regime Gate

SPY is checked on the same 15-min EMA logic. The result is cached for 5 minutes (TTL = 300 s) to avoid hammering the API.

```
async def check_regime_gate(direction):
    spy_trend = await get_cached_spy_trend() # cached for 300 s
    if direction == "CALL" and spy_trend == "bearish":
        return False # market falling – skip bullish entry
    if direction == "PUT" and spy_trend == "bullish":
        return False # market rising – skip bearish entry
    return True
```

Layer 7 — Pre-entry Guards

```
def pre_entry_guards(symbol, db):
    # 1. Earnings blackout
    if within_earnings_blackout(symbol): return False
    # 2. Re-entry cooldown after a loss
    if within_cooldown(symbol, db): return False
    # 3. Daily loss limit per symbol
    if too_many_losses_today(symbol, db): return False
    # 4. Lunch-hour noise filter (11:45 – 12:15 ET by default)
    if not is_in_trading_window(datetime.now()): return False
    return True
```

4 · How Trades Are Exited

Once a trade is open, the manage loop checks several exit conditions on every 30-second tick. The first condition that fires wins.

Reason	Description
STOP	Option price falls to entry $\times$ (1 – 22%). Hard stop — no delay, no override.
TP / TAKE_PROFIT	Option price rises to entry $\times$ (1 + 40%). Full exit at target.
TRAILING_STOP	Once up 9%+, a trailing stop activates. It keeps 50% of the peak gain. E.g. entry \$5, peak \$5.80 → trailing stop at \$5.40
VWAP_BREAK	1-min price crosses back through VWAP against the trade direction — signals the setup has failed.
TREND_REVERSAL	15-min EMA flips against the trade. Suppressed for the first 20 min after entry to avoid choppy-bar noise.
EOD	End-of-day sweep. All open trades are closed at 14:45 ET regardless of P&L.

Trailing Stop Logic

```
def check_trailing_stop(trade, current_price):
    pct_gain = (current_price - trade.entry_price) / trade.entry_price
    if pct_gain < 0.09:
        return None    # not yet in trailing-stop territory

    # Peak gain so far
    peak = trade.peak_price or trade.entry_price
    if current_price > peak:
        trade.peak_price = current_price    # update high-water mark

    # Trailing stop = entry + 50% of (peak - entry)
    trail_stop = trade.entry_price + 0.50 * (trade.peak_price - trade.entry_price)
    if current_price <= trail_stop:
        return "TRAILING_STOP"
    return None
```

5 · Configuration Reference (.env)

All settings live in **.env** at the project root. They are loaded at startup by **app/config.py** using pydantic-settings. Change a value, restart the app — no code changes needed.

Trading Window

Setting	Default	What it does
TRADING_START_TIME	11:15	Don't enter trades before this ET time (let open volatility settle).
TRADING_END_TIME	14:45	Close everything and stop entries after this time.
MAX_OPEN_TRADES	3	Maximum concurrent open positions across all symbols.
LUNCH_BREAK_ENABLED	1	Block entries during lunch (1 = on, 0 = off).
LUNCH_BREAK_START	11:45	ET time when lunch break begins.
LUNCH_BREAK_END	12:15	ET time when lunch break ends.

Position Sizing

Setting	Default	What it does
AMOUNT_PER_TRADE	\$500	USD budget per trade (option premium paid, not notional).
MAX_DAILY_LOSS	\$500	Halt all trading if daily P&L drops below negative this amount.

Risk / Reward

Setting	Default	What it does
STOP_LOSS_PCT	0.22 (22%)	Exit if option price drops 22% below entry.
TAKE_PROFIT_PCT	0.40 (40%)	Exit if option price rises 40% above entry.

Strategy Params

Setting	Default	What it does
EMA_PERIOD	9	Period for the Exponential Moving Average on 15-min bars.
EMA_CONSECUTIVE_BARS	2	How many consecutive 15-min bars must be above/below EMA before entering.
BOUNCE_BARS_REQUIRED	2	How many consecutive 1-min bars must confirm the VWAP bounce.
VWAP_BAND_PCT	0.01	Tolerance around VWAP for a pullback to count (1%).
VWAP_SLOPE_LOOKBACK_BARS20		Compare VWAP now vs this many 1-min bars ago to measure slope.
VWAP_SLOPE_THRESHOLD_PCT0.05		Minimum slope (%) to confirm VWAP has real direction.

Regime & IV Filters

Setting	Default	What it does
REGIME_GATE_ENABLED	1	Enable SPY regime filter (1 = on).
REGIME_GATE_SYMBOL	SPY	Which ETF to use as the market proxy.
REGIME_GATE_TTL_SECONDS	300	Cache SPY trend for this many seconds.
IV_MAX_THRESHOLD	1.50	Block trade if IV > 150% (option too expensive).

Pre-entry Guards

Setting	Default	What it does
EARNINGS_SKIP_DAYS	1	Skip symbol N calendar days before its earnings date.
COOLDOWN_MINUTES	60	Wait this many minutes before re-entering a symbol after a STOP or VWAP_BREAK exit.
MAX_LOSSES_PER_SYMBOL_PER_DAY	2	Stop trading a symbol for the day after this many losing trades.
TREND_REVERSAL_MIN_HOLD_MINUTES	20	Don't allow TREND_REVERSAL exit for this many minutes after entry.

6 · How to Run ajoy

First-time Setup

```
# 1. Clone the repo and enter the folder
git clone <repo-url> ajoy && cd ajoy

# 2. Create a Python virtual environment
python -m venv .venv
source .venv/bin/activate      # macOS / Linux
# .venv\Scripts\activate      # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Copy the environment file and fill in your API keys
cp .env.example .env
# Edit .env: set TRADIER_API_TOKEN, TRADIER_ACCOUNT_ID, etc.

# 5. Run the app (sandbox mode — no real money)
uvicorn app.main:app --reload
```

Running Tests

```
# Run all tests
pytest tests/ -v

# Run a specific test file
pytest tests/test_16_ema_confirm.py -v

# Run tests and see print output
pytest tests/ -v -s
```

Database Reconciliation

If the app restarts mid-session, open trades may be orphaned in the DB. Run the reconciliation script to sync DB state with live broker positions:

```
python scripts/reconcile.py
```

Switching to Live Trading

Change two lines in .env:

```
# Sandbox (safe, no real money):
TRADIER_BASE_URL=https://sandbox.tradier.com/v1
TRADIER_API_TOKEN=<your sandbox token>

# Live (real money — be careful!):
```

```
TRADIER_BASE_URL=https://api.tradier.com/v1  
TRADIER_API_TOKEN=<your live token>
```

7 · Reading the Dashboard

Open **http://localhost:8000** in your browser after starting the app. The dashboard has five tabs:

Tab	What you see
Overview	Daily P&L, win rate, number of trades, current open positions.
Open Trades	Live table of all open trades with entry price, current price, unrealised P&L, and active exit levels.
Trade History	Closed trades with entry/exit prices, P&L, duration, and exit reason.
Config	Read-only view of all settings loaded from .env (handy for sanity-checks).
Logs	Live tail of the application log — useful for debugging why a trade was or wasn't taken.

8 · File Structure

```

ajoy/
├── app/
│   ├── main.py           ← FastAPI app, scheduler startup
│   ├── config.py         ← All settings (loaded from .env)
│   ├── models.py         ← SQLAlchemy ORM models (Trade, Order)
│   ├── database.py       ← Async DB session factory
│   ├── routers/
│   │   ├── dashboard.py  ← REST endpoints for the UI
│   │   └── services/
│   │       ├── strategy.py ← Entry gates, exit logic, indicators
│   │       ├── tradier.py  ← Tradier API client (bars, quotes, orders)
│   │       ├── options.py  ← Option chain selection + delta filter
│   │       └── regime.py   ← SPY trend cache (Layer 5)
│   └── tests/
│       ├── conftest.py    ← Shared fixtures (make_bar, rising_bars ...)
│       ├── test_01_*.py   ← Unit tests for each feature
│       └── ...
├── scripts/
│   └── reconcile.py       ← Sync DB with live broker positions
├── .env                  ← Your API keys + settings (never commit!)
├── requirements.txt
└── README.md

```

Key File: `app/services/strategy.py`

This is the heart of the bot. It contains every indicator function and every entry/exit decision. If you want to understand or change the trading logic, start here.

Important functions:

Function	Purpose
<code>calculate_ema()</code>	Compute EMA for a list of closes.
<code>calculate_vwap()</code>	Compute VWAP from a list of bars.
<code>ema_direction()</code>	Single-point check: is price above or below EMA? Used by exits and UI.
<code>is_ema_trend_confirmed()</code>	Multi-bar EMA check for entries: were the last N bars all on the correct side?
<code>check_bounce_confirmation()</code>	Did the last N 1-min bars bounce off VWAP in the right direction?
<code>check_entry_signal()</code>	Runs all 7 entry gates. Returns a Signal object or None.
<code>check_exit_signal()</code>	Checks all exit conditions for an open trade. Returns the exit reason or None.
<code>compute_trade_levels()</code>	Computes stop price and take-profit price from entry price.
<code>is_in_trading_window()</code>	Is the current time inside the allowed trading window (including lunch break)?

Appendix · Glossary

Quick-reference definitions for terms used throughout this guide.

Term	Definition
EMA	Exponential Moving Average. A line that smooths out price data, giving more weight to recent bars. A rising EMA = uptrend.
VWAP	Volume-Weighted Average Price. The average price weighted by volume. Institutions use it as a reference — pullbacks to VWAP = good.
CALL	An option contract that gains value when the underlying stock goes UP.
PUT	An option contract that gains value when the underlying stock goes DOWN.
Delta	How much the option price moves for a \$1 move in the stock. Delta 0.40 = option moves ~\$0.40 per \$1 stock move.
IV	Implied Volatility. The market's expectation of future price swings, expressed as an annualised %. High IV = expensive options.
OHLCV	Open, High, Low, Close, Volume. The five data points that make up a candlestick bar.
Trailing Stop	A stop that moves up (for CALLs) as the trade gains — locking in profit while letting winners run.
PCR	Put/Call Ratio. PCR > 1 = more puts than calls = bearish sentiment. PCR < 1 = bullish sentiment.
Regime	The overall market direction (up/down/flat) used to filter trades. ajoy uses SPY as the regime proxy.
Layer / Gate	One of the 7 conditions that must all be True before an entry is allowed.
Cooldown	A waiting period after a losing trade before the same symbol can be entered again.
EOD	End of Day. ajoy closes all positions at 14:45 ET to avoid overnight risk.